

Iteration Expressions

SSA is a useful IR because it enables complex, whole-procedure transformation of code, but its block-statement-value hierarchy makes it difficult to uniformly formulate the allowed transformations as an (in)equational theory. Instead, therefore, we will start by studying a simple, first-order language of expressions, λ_{iter} , equipped with an *iteration operator* – we will then show that this language is *equivalent* to SSA, and therefore we'll be able to use it as a tool to more effectively give a sound and complete equational, and later refinement, theory for SSA.

0.1. Syntax

λ_{iter} is a standard first-order expression language with branching and iteration: a functional analogue of WHILE. Its grammar is parametrized by a set of *base types* $X \in \mathcal{X}$ and a set of *instructions* $f \in \mathcal{J}$. To model multiple arguments and control flow, our grammar of types A, B, C includes all *tensor products* $A \otimes B$ and *coproducts* $A + B$ generated by base types X , along with a *unit type* $\mathbf{1}$ and an *empty type* $\mathbf{0}$. Mirroring this, expressions a, b, c consist of

- *Variables* x, y, z
- *Applications* $f a$, consisting of instructions $f \in \mathcal{J}$ applied to an expression a
- *Let-bindings* $\text{let } x = a \text{ } b$ and *destructuring let-bindings* $\text{let } (x, y) = a \text{ } b$. We write $a; b$ as syntactic sugar for $\text{let } \cdot = a \text{ } b$ (i.e., a let-binding where the bound variable is not used in b).
- *Case-expressions* $\text{case } e \{ \iota_l x : a, \iota_r y : b \}$, representing branching control-flow
- *Iteration-expressions* $\text{iter } a \{ \iota_r x : b \}$, representing a variant of *tail-controlled loop*, which:
 - Evaluates an initial value $a : A$
 - Evaluates the loop body $b : B + A$ with $x = a$; if the loop evaluates to a value of type B , we return it, otherwise, we re-evaluate the loop with x having the new value of type A

We give the full grammar for λ_{iter} types, expressions, and contexts in Figure 1.

$$\begin{aligned}
 A, B, C &::= X \mid A \otimes B \mid \mathbf{1} \mid A + B \mid \mathbf{0} \\
 a, b, c &::= x \mid f a \mid \text{let } x = a \text{ } b \mid () \mid (a, b) \mid \text{let } (x, y) = a \text{ } b \mid \iota_l a \mid \iota_r b \mid \text{case } a \{ \iota_l x : \\
 &\quad b, \iota_r y : c \} \mid \text{abort } a \mid \text{iter } a \{ \iota_r x : b \} \\
 \Gamma &::= \cdot \mid \Gamma, x : A
 \end{aligned}$$

Figure 1: Syntax for λ_{iter} types, expressions, and contexts.

We can then give typing rules for λ_{iter} as follows: **TODO 1: give raw typing rules**

TODO 2: weakening

TODO 3: substitution

TODO 4: